

backend.md

Backend API Server 需求、設計與實作計劃

1. 目的

Backend API Server 是 `structure.md` 所描述之業務作業系統的受治理控制層。

其目的是透過安全的 API 對外暴露系統的內部能力，讓前端應用程式、管理主控台、CLI、自動化工具或外部整合，可以安全地與 agents、workflows、knowledge、memory、governance、evaluations、audit logs 及 process intelligence 互動。

前端不得直接存取 agents、databases、workflow engines、LLM providers、vector stores 或內部工具。所有存取都必須經由 Backend API Server。

簡而言之：

```
Frontend = 使用者體驗層
Backend API = 受治理的智能與控制層
Agents = 專門化工作者
Workflows = 結構化業務執行路徑
Governance = 風險與審批控制
Audit = 信任與可追溯層
```

2. 主要目標

建立一個 backend server，封裝整個系統架構的完整功能，並透過版本化 API 對外提供。

backend 應支援：

- 安全驗證
 - 以角色為基礎的存取控制
 - agent orchestration
 - workflow 執行
 - workflow run 追蹤
 - governance 審批關卡
 - memory 與 knowledge 檢索
 - audit logging
 - evaluation 與品質檢查
 - process intelligence APIs
 - 背景 workers
 - 向前端串流更新
 - 可擴展的 integrations
-

3. 建議技術棧

本文件建議使用以下技術棧：

```
Backend Framework: FastAPI
Language: Python
Database: PostgreSQL
Vector Search: pgvector, Qdrant, Weaviate, or Pinecone
Queue: Redis + Celery/RQ/Arq, or Temporal for advanced workflows
Cache: Redis
Object Storage: S3-compatible storage
LLM Provider Layer: Provider-agnostic abstraction
Auth: JWT / OAuth2 / SSO-ready
API Style: REST first, WebSocket/SSE for streaming
Documentation: OpenAPI generated from FastAPI
Containerization: No Docker
Deployment: No Docker
```

亦可接受其他 backend 技術棧，例如使用 Node.js 配合 NestJS，但整體架構應維持一致。

4. 系統邊界

Backend API Server 擁有並控制以下範圍：

- API access
- user authentication
- authorization
- agent execution requests
- workflow execution requests
- governance checks
- approval gates
- audit events
- knowledge access
- memory access
- process intelligence
- evaluations
- background jobs
- integrations

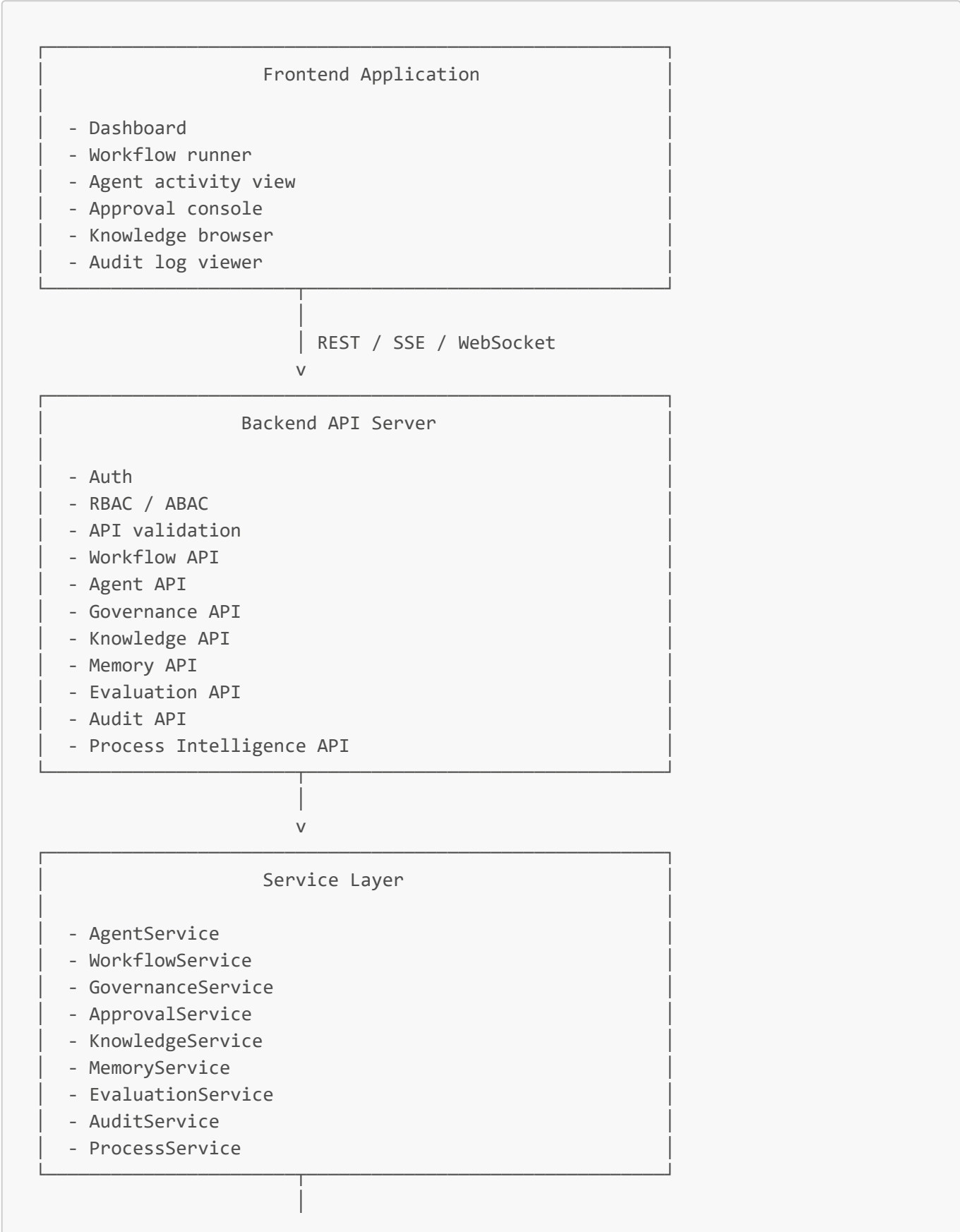
Backend API Server 不得只是一個盲目把請求轉發給 agents 的薄代理層。

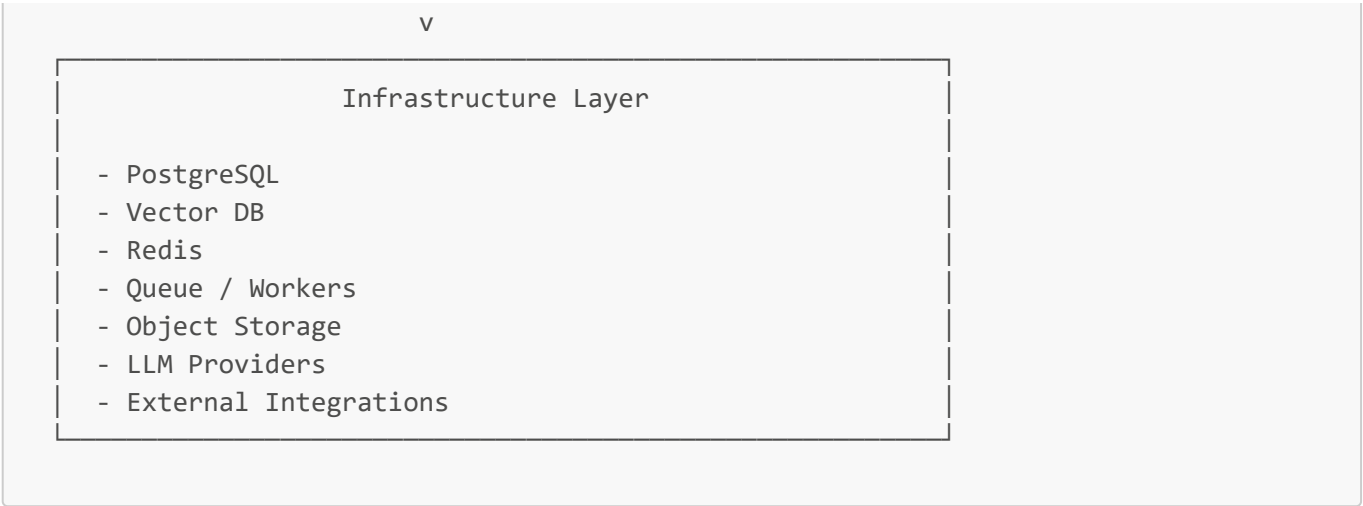
相反，它必須強制執行：

- permissions
- policy
- workflow rules
- risk controls

- auditability
- data access boundaries
- evaluation checks

5. 高層架構





6. 核心設計原則

6.1 API First

所有平台功能都應可透過已文件化的 API 存取。

前端應只依賴 backend API 合約，而不是依賴內部實作細節。

6.2 預設安全

除非明確標示為公開，否則每個 endpoint 都必須要求驗證。

每個敏感操作都必須檢查授權。

每次 workflow 執行都必須可供審計。

6.3 Governance First

在 agent 或 workflow 執行之前，backend 必須先決定某項操作是否被允許。

governance 檢查應包括：

- user permissions
- workflow permissions
- data access permissions
- risk level
- approval requirement
- tool access permissions
- policy compliance

6.4 Human-in-the-Loop

高風險操作在執行前必須暫停，並要求人工審批。

例子：

- sending emails externally
- modifying customer records
- deleting data
- publishing content
- approving financial actions
- changing workflow definitions
- changing governance policies

6.5 為所有重要事項建立審計

每個重要操作都必須產生 audit event。

在可行情況下，audit logs 應採用 append-only。

6.6 以 Workers 處理長時間任務

agent workflows 不得完全在一般 API request thread 內執行。

相反：

```
Frontend -> Backend API -> Queue -> Worker -> Database -> Stream updates to frontend
```

6.7 前端簡化

前端只需要知道：

- what APIs exist
- what input is required
- what status is returned
- where to stream progress
- where to fetch results

前端不應知道內部 agent routing、LLM provider 邏輯、memory retrieval 內部實作或 governance policy 細節。

7. 功能需求

7.1 Authentication

backend 必須支援已驗證使用者。

最低要求：

- user registration or admin-created users
- login
- logout
- token refresh
- password reset if password auth is used
- API key support for machine clients
- optional SSO/OAuth support

建議的驗證方式：

- JWT access tokens
- refresh tokens
- API keys for service integrations
- OAuth2/OIDC compatibility

7.2 Authorization

backend 必須支援以角色為基礎的存取控制。

建議角色：

```
Owner
Admin
Manager
Operator
Reviewer
Viewer
ServiceAccount
```

權限群組範例：

```
agents:read
agents:create
agents:update
agents:delete
workflows:read
workflows:create
workflows:update
workflows:run
workflow_runs:read
workflow_runs:cancel
```

```
knowledge:read
knowledge:write
memory:read
memory:write
governance:read
governance:update
approvals:read
approvals:approve
approvals:reject
audit:read
evaluations:read
settings:update
```

授權系統應支援未來的 ABAC 規則，例如：

```
user.department == resource.department
user.organization_id == resource.organization_id
workflow.risk_level <= user.max_risk_level
```

7.3 使用者與組織管理

backend 應支援：

- organizations
- users
- teams
- roles
- permissions
- invitations
- service accounts
- API keys

即使系統初期是單租戶，資料庫設計仍應包含 `organization_id` 或 `tenant_id`，以便未來更容易支援多租戶。

7.4 Agent Registry

backend 必須維護可用 agents 的 registry。

每個 agent 應定義：

- agent ID
- name
- description
- version

- owner
- department
- allowed tools
- allowed memory scopes
- allowed workflow types
- risk level
- input schema
- output schema
- runtime configuration
- status

Agent 狀態：

```
draft
active
disabled
archived
```

前端應能列出 agents、檢視 agent metadata，以及查看 agent activity。

7.5 Tool Registry

agents 可使用 tools，但只能透過 backend 控制的權限來使用。

每個 tool 應定義：

- tool ID
- name
- description
- category
- input schema
- output schema
- risk level
- required permissions
- approval requirement
- timeout
- retry policy
- enabled/disabled status

Tool 類別可包括：

```
database
email
calendar
crm
file
```



```
web
internal_api
external_api
llm
code_execution
human_approval
```

高風險工具必須要求明確的 governance 規則。

7.6 Workflow Management

backend 必須支援 workflow definitions。

workflow 代表一個結構化業務流程，可能包括：

- one or more agents
- multiple steps
- conditions
- approvals
- memory retrieval
- tool execution
- evaluation checks
- final output generation

Workflow metadata 應包括：

- workflow ID
- name
- description
- version
- owner
- department
- risk level
- input schema
- output schema
- steps
- governance policy
- evaluation policy
- status

Workflow 狀態：

```
draft
active
disabled
archived
```

7.7 Workflow Run Management

每次 workflow 執行都必須建立一個 `workflow_run`。

Workflow run 狀態：

```
queued
running
waiting_for_approval
paused
completed
failed
cancelled
expired
```

每個 workflow run 應儲存：

```
- workflow_run ID
- workflow ID
- workflow version
- requested by user
- organization ID
- input
- status
- current step
- output
- error
- timestamps
- token/cost usage
- approval state
- evaluation results
```

前端必須能夠：

```
- start a workflow run
- view run status
- view run steps
- cancel a run
- retry a failed run if allowed
- stream progress
- view final output
```

7.8 Workflow Step Tracking

每個 workflow run 應包含 step records。

Step 狀態：

```
pending
running
waiting_for_approval
completed
failed
skipped
cancelled
```

每個 step 應儲存：

```
- step ID
- workflow_run ID
- step name
- step type
- agent ID if applicable
- tool ID if applicable
- input
- output
- error
- started_at
- completed_at
- duration
- retry_count
```

7.9 Governance 與 Approval System

backend 必須包含 governance layer。

governance system 負責決定：

```
- whether a workflow can run
- whether a step can execute
- whether a tool can be used
- whether data can be accessed
- whether human approval is required
- whether output can be released
```

Approval request 狀態：

```
pending
approved
```

```
rejected
expired
cancelled
```

Approval requests 應儲存：

```
- approval ID
- workflow_run ID
- step ID
- requested action
- risk level
- requested by
- assigned reviewer
- decision
- decision reason
- created_at
- decided_at
```

7.10 Knowledge Base

backend 必須對外提供受控的 knowledge 存取。

knowledge 來源可包括：

```
- uploaded documents
- markdown files
- PDFs
- internal policies
- process documentation
- database records
- support articles
- SOPs
- meeting notes
```

Knowledge 功能：

```
- upload document
- index document
- chunk document
- embed chunks
- search documents
- retrieve document metadata
- delete or archive documents
- enforce ACL permissions
```

Document 狀態：

```
uploaded
processing
indexed
failed
archived
deleted
```

7.11 Memory System

backend 必須管理 agent 與 workflow memory。

Memory 類型：

```
short_term
long_term
user_memory
team_memory
department_memory
organization_memory
workflow_memory
agent_memory
```

Memory 必須具備 access control。

規則示例：

```
Sales Agent can access sales memory.
Finance Agent can access finance memory.
HR Agent can access HR memory.
General agents cannot access restricted department memory.
```

Memory entries 應包括：

- memory ID
- scope
- owner
- organization ID
- department
- content
- metadata
- embedding reference
- sensitivity level

- expiration date
- created_at

7.12 Evaluation System

backend 應在回傳或釋出重要輸出前先進行評估。

Evaluation 類型：

- schema validation
- business rule validation
- policy compliance
- hallucination risk check
- completeness check
- formatting check
- safety check
- cost check
- human review

Evaluation result 狀態：

```
passed
failed
warning
requires_review
```

Evaluation results 應關聯至：

- workflow run
- workflow step
- agent output
- final output

7.13 Audit Logging

backend 必須為重要操作產生 audit logs。

Audit events 應包括：

- user login
- workflow created
- workflow updated
- workflow run started

- workflow run completed
- workflow run failed
- approval requested
- approval approved
- approval rejected
- knowledge uploaded
- knowledge deleted
- memory accessed
- memory updated
- governance rule changed
- agent tool used
- API key created
- permission changed

Audit log 欄位：

- audit ID
- organization ID
- actor user ID
- actor type
- action
- resource type
- resource ID
- request ID
- IP address
- user agent
- before state
- after state
- metadata
- status
- created_at

7.14 Process Intelligence

backend 應對外提供 process intelligence APIs。

process intelligence 可包括：

- workflow performance analytics
- bottleneck detection
- failure patterns
- average duration
- approval delays
- agent performance
- cost by workflow
- cost by department
- task success rate
- automation opportunities

初版 process APIs 可以只先對 workflow run 資料做聚合。

7.15 串流更新

backend 應支援即時進度更新。

建議選項：

```
Server-Sent Events for simple one-way updates
WebSocket for two-way interactive agent sessions
```

對於第一版而言，Server-Sent Events 通常更簡單。

串流 event 類型：

```
run.started
run.status_changed
step.started
step.completed
step.failed
approval.requested
approval.approved
approval.rejected
evaluation.completed
run.completed
run.failed
log.message
```

串流 event 範例：

```
{
  "event": "step.completed",
  "workflow_run_id": "run_123",
  "step_id": "step_456",
  "status": "completed",
  "message": "Knowledge retrieval completed",
  "timestamp": "2026-07-07T12:00:00Z"
}
```

8. 非功能需求

8.1 Security

backend 必須：

- require authentication for protected routes
- enforce authorization on every protected resource
- validate all request bodies
- sanitize file uploads
- protect secrets
- rate limit sensitive endpoints
- log security-relevant events
- avoid leaking internal errors
- encrypt sensitive data where required
- protect against prompt injection in agent workflows

8.2 Reliability

backend 應：

- use background workers for long tasks
- retry recoverable failures
- use idempotency keys for workflow creation
- persist workflow state
- handle worker crashes
- allow workflow recovery where possible
- expose health checks

8.3 Scalability

backend 應被設計為以下元件可以獨立擴展：

- API server
- worker processes
- database
- vector store
- cache
- queue

8.4 Observability

backend 必須提供：

- structured logs
- request IDs

- audit logs
- metrics
- traces if possible
- workflow run history
- worker job logs
- error reports

重要 metrics :

- API request latency
- workflow run duration
- workflow failure rate
- queue depth
- approval wait time
- LLM token usage
- LLM cost
- database latency
- vector search latency
- active users

8.5 Maintainability

backend 應採用分層架構 :

```
API routes -> Services -> Domain logic -> Infrastructure adapters
```

Routes 必須保持精簡。

業務邏輯應放在 services 與 domain modules。

外部 providers 應透過 interfaces 抽象化。

9. 建議的 Backend 資料夾結構

```
backend/  
  app/  
    main.py  
  
  api/  
    __init__.py  
    dependencies.py  
    errors.py  
    v1/  
      __init__.py
```

```
router.py
routes/
  auth.py
  users.py
  organizations.py
  agents.py
  tools.py
  workflows.py
  workflow_runs.py
  approvals.py
  governance.py
  knowledge.py
  memory.py
  evaluations.py
  audit_logs.py
  processes.py
  settings.py
  health.py

core/
  __init__.py
  config.py
  security.py
  auth.py
  permissions.py
  logging.py
  errors.py
  pagination.py
  idempotency.py
  rate_limit.py

domain/
  __init__.py
  agents/
    models.py
    policies.py
    runtime.py
  workflows/
    models.py
    engine.py
    states.py
    policies.py
  governance/
    models.py
    policy_engine.py
    risk.py
  approvals/
    models.py
    service.py
  knowledge/
    models.py
    chunking.py
    retrieval.py
  memory/
```

```
models.py
scopes.py
retrieval.py
evaluations/
  models.py
  evaluators.py
audit/
  models.py
  events.py
processes/
  analytics.py

services/
  __init__.py
  auth_service.py
  user_service.py
  organization_service.py
  agent_service.py
  tool_service.py
  workflow_service.py
  workflow_run_service.py
  governance_service.py
  approval_service.py
  knowledge_service.py
  memory_service.py
  evaluation_service.py
  audit_service.py
  process_service.py
  notification_service.py

infrastructure/
  __init__.py
  database/
    __init__.py
    session.py
    models.py
    migrations/
  repositories/
    user_repository.py
    agent_repository.py
    workflow_repository.py
    workflow_run_repository.py
    approval_repository.py
    knowledge_repository.py
    memory_repository.py
    audit_repository.py
  vector_store/
    __init__.py
    base.py
    pgvector_store.py
    qdrant_store.py
  object_storage/
    __init__.py
    base.py
```

```
s3_storage.py
local_storage.py
queue/
  __init__.py
  broker.py
  tasks.py
llm/
  __init__.py
  base.py
  openai_provider.py
  mock_provider.py
integrations/
  __init__.py
  email.py
  crm.py
  calendar.py

schemas/
  __init__.py
  common.py
  auth.py
  users.py
  organizations.py
  agents.py
  tools.py
  workflows.py
  workflow_runs.py
  approvals.py
  governance.py
  knowledge.py
  memory.py
  evaluations.py
  audit_logs.py
  processes.py

workers/
  __init__.py
  workflow_worker.py
  knowledge_worker.py
  evaluation_worker.py
  memory_worker.py

tests/
  unit/
  integration/
  e2e/
  security/
  load/

scripts/
  seed.py
  create_admin.py
  migrate.py
```

```
pyproject.toml
README.md
backend.md
```

10. 資料模型

10.1 核心實體

系統應包含以下主要實體：

```
Organization
User
Role
Permission
APIKey
Agent
Tool
Workflow
WorkflowVersion
WorkflowRun
WorkflowRunStep
ApprovalRequest
GovernancePolicy
KnowledgeDocument
KnowledgeChunk
MemoryItem
EvaluationRun
AuditLog
ProcessMetric
Notification
```

10.2 Organization

Field	Type	Required	Notes
id	UUID	Yes	Primary key
name	String	Yes	Organization name
slug	String	Yes	Unique slug
status	String	Yes	active, disabled
settings	JSON	No	Organization settings
created_at	DateTime	Yes	Creation time
updated_at	DateTime	Yes	Update time

10.3 User

Field	Type	Required	Notes
id	UUID	Yes	Primary key
organization_id	UUID	Yes	Tenant boundary
email	String	Yes	Unique per organization
name	String	Yes	Display name
password_hash	String	No	Required only for password auth
status	String	Yes	active, invited, disabled
role_id	UUID	Yes	Primary role
department	String	No	Optional department
last_login_at	DateTime	No	Last login
created_at	DateTime	Yes	Creation time
updated_at	DateTime	Yes	Update time

10.4 Agent

Field	Type	Required	Notes
id	UUID	Yes	Primary key
organization_id	UUID	Yes	Tenant boundary
name	String	Yes	Agent name
description	Text	No	Agent description
version	String	Yes	Agent version
status	String	Yes	draft, active, disabled, archived
owner_user_id	UUID	No	Owner
department	String	No	Department scope
allowed_tools	JSON	Yes	Tool IDs or permission rules
allowed_memory_scopes	JSON	Yes	Memory access rules
input_schema	JSON	No	JSON schema
output_schema	JSON	No	JSON schema
runtime_config	JSON	No	Model, temperature, limits
risk_level	String	Yes	low, medium, high, critical

Field	Type	Required	Notes
created_at	DateTime	Yes	Creation time
updated_at	DateTime	Yes	Update time

10.5 Workflow

Field	Type	Required	Notes
id	UUID	Yes	Primary key
organization_id	UUID	Yes	Tenant boundary
name	String	Yes	Workflow name
description	Text	No	Workflow description
status	String	Yes	draft, active, disabled, archived
current_version_id	UUID	No	Active version
owner_user_id	UUID	No	Owner
department	String	No	Department scope
risk_level	String	Yes	low, medium, high, critical
created_at	DateTime	Yes	Creation time
updated_at	DateTime	Yes	Update time

10.6 WorkflowVersion

Field	Type	Required	Notes
id	UUID	Yes	Primary key
workflow_id	UUID	Yes	Parent workflow
version	String	Yes	Version label
input_schema	JSON	No	Input validation
output_schema	JSON	No	Output validation
steps	JSON	Yes	Workflow DNA
governance_policy	JSON	No	Approval/risk rules
evaluation_policy	JSON	No	Output checks
created_by_user_id	UUID	Yes	Version creator
created_at	DateTime	Yes	Creation time

10.7 WorkflowRun

Field	Type	Required	Notes
id	UUID	Yes	Primary key
organization_id	UUID	Yes	Tenant boundary
workflow_id	UUID	Yes	Workflow
workflow_version_id	UUID	Yes	Exact version executed
requested_by_user_id	UUID	Yes	User who started run
status	String	Yes	queued, running, waiting_for_approval, completed, failed, cancelled
input	JSON	Yes	Submitted input
output	JSON	No	Final output
error	JSON	No	Failure info
current_step_id	UUID	No	Current step
idempotency_key	String	No	Prevent duplicate runs
cost_summary	JSON	No	Token/cost usage
started_at	DateTime	No	Start time
completed_at	DateTime	No	End time
created_at	DateTime	Yes	Creation time
updated_at	DateTime	Yes	Update time

10.8 WorkflowRunStep

Field	Type	Required	Notes
id	UUID	Yes	Primary key
workflow_run_id	UUID	Yes	Parent run
step_key	String	Yes	Step key from workflow definition
name	String	Yes	Step name
type	String	Yes	agent, tool, approval, evaluation, condition
status	String	Yes	pending, running, completed, failed, skipped
agent_id	UUID	No	Agent used
tool_id	UUID	No	Tool used
input	JSON	No	Step input

Field	Type	Required	Notes
output	JSON	No	Step output
error	JSON	No	Step error
retry_count	Integer	Yes	Retry count
started_at	DateTime	No	Start time
completed_at	DateTime	No	End time
created_at	DateTime	Yes	Creation time

10.9 ApprovalRequest

Field	Type	Required	Notes
id	UUID	Yes	Primary key
organization_id	UUID	Yes	Tenant boundary
workflow_run_id	UUID	Yes	Parent run
workflow_run_step_id	UUID	No	Step requiring approval
requested_by_user_id	UUID	Yes	Requesting user
reviewer_user_id	UUID	No	Assigned reviewer
status	String	Yes	pending, approved, rejected, expired, cancelled
risk_level	String	Yes	low, medium, high, critical
requested_action	JSON	Yes	Action to approve
reason	Text	No	Reason approval is needed
decision_reason	Text	No	Reviewer notes
decided_at	DateTime	No	Decision time
created_at	DateTime	Yes	Creation time

10.10 KnowledgeDocument

Field	Type	Required	Notes
id	UUID	Yes	Primary key
organization_id	UUID	Yes	Tenant boundary
title	String	Yes	Document title
source_type	String	Yes	upload, url, database, manual
storage_uri	String	No	Object storage path

Field	Type	Required	Notes
status	String	Yes	uploaded, processing, indexed, failed, archived
sensitivity	String	Yes	public, internal, confidential, restricted
acl	JSON	No	Access rules
metadata	JSON	No	Document metadata
uploaded_by_user_id	UUID	No	Uploader
created_at	DateTime	Yes	Creation time
updated_at	DateTime	Yes	Update time

10.11 KnowledgeChunk

Field	Type	Required	Notes
id	UUID	Yes	Primary key
document_id	UUID	Yes	Parent document
organization_id	UUID	Yes	Tenant boundary
chunk_index	Integer	Yes	Order
content	Text	Yes	Chunk text
embedding_id	String	No	Vector DB reference
metadata	JSON	No	Chunk metadata
created_at	DateTime	Yes	Creation time

10.12 MemoryItem

Field	Type	Required	Notes
id	UUID	Yes	Primary key
organization_id	UUID	Yes	Tenant boundary
scope	String	Yes	user, team, department, organization, agent, workflow
owner_id	UUID	No	User/agent/workflow owner
department	String	No	Department scope
content	Text	Yes	Memory content
metadata	JSON	No	Metadata
sensitivity	String	Yes	public, internal, confidential, restricted
expires_at	DateTime	No	Expiration

Field	Type	Required	Notes
created_by	UUID	No	Creator
created_at	DateTime	Yes	Creation time

10.13 EvaluationRun

Field	Type	Required	Notes
id	UUID	Yes	Primary key
organization_id	UUID	Yes	Tenant boundary
workflow_run_id	UUID	No	Parent run
workflow_run_step_id	UUID	No	Parent step
evaluator_type	String	Yes	schema, policy, llm, human, rule
status	String	Yes	passed, failed, warning, requires_review
score	Float	No	Numeric score
input	JSON	No	Evaluation input
output	JSON	No	Evaluation output
reason	Text	No	Explanation
created_at	DateTime	Yes	Creation time

10.14 AuditLog

Field	Type	Required	Notes
id	UUID	Yes	Primary key
organization_id	UUID	Yes	Tenant boundary
actor_user_id	UUID	No	User actor
actor_type	String	Yes	user, agent, service_account, system
action	String	Yes	Action name
resource_type	String	Yes	Resource type
resource_id	UUID	No	Resource ID
request_id	String	No	Request correlation
ip_address	String	No	Client IP
user_agent	String	No	Client user agent
before_state	JSON	No	Before change

Field	Type	Required	Notes
after_state	JSON	No	After change
metadata	JSON	No	Extra metadata
status	String	Yes	success, failure
created_at	DateTime	Yes	Creation time

11. API 設計

11.1 API 版本控制

所有公開 API 都必須版本化。

```
/api/v1/...
```

未來如有破壞性變更，應使用：

```
/api/v2/...
```

11.2 回應格式

成功回應範例：

```
{
  "data": {
    "id": "example_id"
  },
  "meta": {
    "request_id": "req_123"
  }
}
```

列表回應範例：

```
{
  "data": [],
  "pagination": {
    "page": 1,
    "page_size": 25,
    "total": 100
  }
}
```

```
    },
    "meta": {
      "request_id": "req_123"
    }
  }
}
```

錯誤回應範例：

```
{
  "error": {
    "code": "permission_denied",
    "message": "You do not have permission to run this workflow.",
    "details": {}
  },
  "meta": {
    "request_id": "req_123"
  }
}
```

11.3 常用錯誤代碼

```
bad_request
unauthorized
permission_denied
not_found
validation_error
conflict
rate_limited
approval_required
workflow_not_active
workflow_run_failed
agent_not_available
tool_not_allowed
knowledge_access_denied
memory_access_denied
evaluation_failed
internal_error
```

11.4 Authentication Endpoints

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Login
POST	/api/v1/auth/logout	Logout

Method	Endpoint	Purpose
POST	/api/v1/auth/refresh	Refresh token
GET	/api/v1/auth/me	Current user
POST	/api/v1/auth/api-keys	Create API key
GET	/api/v1/auth/api-keys	List API keys
DELETE	/api/v1/auth/api-keys/{key_id}	Revoke API key

11.5 Agent Endpoints

Method	Endpoint	Purpose
GET	/api/v1/agents	List agents
POST	/api/v1/agents	Create agent
GET	/api/v1/agents/{agent_id}	Get agent
PATCH	/api/v1/agents/{agent_id}	Update agent
DELETE	/api/v1/agents/{agent_id}	Archive agent
GET	/api/v1/agents/{agent_id}/activity	Agent activity
GET	/api/v1/agents/{agent_id}/tools	Allowed tools

11.6 Workflow Endpoints

Method	Endpoint	Purpose
GET	/api/v1/workflows	List workflows
POST	/api/v1/workflows	Create workflow
GET	/api/v1/workflows/{workflow_id}	Get workflow
PATCH	/api/v1/workflows/{workflow_id}	Update workflow
DELETE	/api/v1/workflows/{workflow_id}	Archive workflow
POST	/api/v1/workflows/{workflow_id}/versions	Create version
GET	/api/v1/workflows/{workflow_id}/versions	List versions
POST	/api/v1/workflows/{workflow_id}/activate	Activate workflow
POST	/api/v1/workflows/{workflow_id}/disable	Disable workflow

11.7 Workflow Run Endpoints

Method	Endpoint	Purpose
--------	----------	---------

Method	Endpoint	Purpose
POST	/api/v1/workflows/{workflow_id}/runs	Start workflow run
GET	/api/v1/workflow-runs	List workflow runs
GET	/api/v1/workflow-runs/{run_id}	Get run
GET	/api/v1/workflow-runs/{run_id}/steps	Get run steps
POST	/api/v1/workflow-runs/{run_id}/cancel	Cancel run
POST	/api/v1/workflow-runs/{run_id}/retry	Retry run
GET	/api/v1/workflow-runs/{run_id}/stream	Stream run events

啟動 workflow 請求範例：

```
{
  "input": {
    "customer_id": "cust_123",
    "task": "analyze onboarding process"
  }
}
```

啟動 workflow 回應範例：

```
{
  "data": {
    "workflow_run_id": "run_123",
    "status": "queued"
  },
  "meta": {
    "request_id": "req_123"
  }
}
```

11.8 Approval Endpoints

Method	Endpoint	Purpose
GET	/api/v1/approvals	List approval requests
GET	/api/v1/approvals/{approval_id}	Get approval request
POST	/api/v1/approvals/{approval_id}/approve	Approve request
POST	/api/v1/approvals/{approval_id}/reject	Reject request
POST	/api/v1/approvals/{approval_id}/reassign	Reassign reviewer

批准請求範例：

```
{
  "decision_reason": "Approved because the output was reviewed and is safe to send."
}
```

拒絕請求範例：

```
{
  "decision_reason": "Rejected because customer data appears incomplete."
}
```

11.9 Knowledge Endpoints

Method	Endpoint	Purpose
GET	<code>/api/v1/knowledge/documents</code>	List documents
POST	<code>/api/v1/knowledge/documents</code>	Upload/create document
GET	<code>/api/v1/knowledge/documents/{document_id}</code>	Get document
DELETE	<code>/api/v1/knowledge/documents/{document_id}</code>	Archive/delete document
POST	<code>/api/v1/knowledge/documents/{document_id}/index</code>	Start indexing
POST	<code>/api/v1/knowledge/search</code>	Search knowledge

Knowledge 搜尋範例：

```
{
  "query": "customer onboarding process",
  "filters": {
    "department": "sales",
    "sensitivity": "internal"
  },
  "limit": 10
}
```

11.10 Memory Endpoints

Method	Endpoint	Purpose
GET	<code>/api/v1/memory</code>	List memory items

Method	Endpoint	Purpose
POST	/api/v1/memory	Create memory
GET	/api/v1/memory/{memory_id}	Get memory
PATCH	/api/v1/memory/{memory_id}	Update memory
DELETE	/api/v1/memory/{memory_id}	Delete memory
POST	/api/v1/memory/search	Search memory

11.11 Governance Endpoints

Method	Endpoint	Purpose
GET	/api/v1/governance/policies	List policies
POST	/api/v1/governance/policies	Create policy
GET	/api/v1/governance/policies/{policy_id}	Get policy
PATCH	/api/v1/governance/policies/{policy_id}	Update policy
DELETE	/api/v1/governance/policies/{policy_id}	Archive policy
POST	/api/v1/governance/check	Run policy check

11.12 Evaluation Endpoints

Method	Endpoint	Purpose
GET	/api/v1/evaluations	List evaluations
GET	/api/v1/evaluations/{evaluation_id}	Get evaluation
POST	/api/v1/evaluations/run	Run evaluation manually
GET	/api/v1/workflow-runs/{run_id}/evaluations	Get run evaluations

11.13 Audit Log Endpoints

Method	Endpoint	Purpose
GET	/api/v1/audit-logs	Search audit logs
GET	/api/v1/audit-logs/{audit_id}	Get audit event

Audit logs 應只可透過 API 以唯讀方式存取。

11.14 Process Intelligence Endpoints

Method	Endpoint	Purpose
GET	/api/v1/processes/metrics	Process metrics
GET	/api/v1/processes/workflow-performance	Workflow performance
GET	/api/v1/processes/bottlenecks	Bottleneck detection
GET	/api/v1/processes/costs	Cost analysis
GET	/api/v1/processes/failures	Failure analysis

12. Workflow 執行設計

12.1 Workflow Run 生命週期

```
queued
-> running
  -> waiting_for_approval
    -> running
      -> completed

queued
-> running
  -> failed

queued
-> cancelled

running
-> cancelled
```

12.2 Workflow 啟動流程

```
1. Frontend sends POST /api/v1/workflows/{workflow_id}/runs
2. Backend authenticates user
3. Backend checks user permission
4. Backend loads workflow
5. Backend validates input schema
6. Backend runs governance pre-check
7. Backend creates workflow_run with status queued
8. Backend writes audit log
9. Backend enqueues worker job
10. Backend returns workflow_run_id
11. Worker executes workflow
12. Frontend polls or streams status
```

12.3 Worker 執行流程

1. Worker receives workflow_run_id
2. Worker loads workflow run
3. Worker marks run as running
4. Worker loads workflow version
5. Worker iterates through steps
6. For each step:
 - check cancellation
 - check permissions
 - check governance
 - request approval if required
 - execute agent/tool/evaluation/condition
 - save step output
 - emit event
 - write audit log
7. Run final evaluation
8. Save final output
9. Mark run completed or failed
10. Emit final event

12.4 Step 類型

支援的 workflow step 類型：

```
agent
tool
approval
condition
knowledge_search
memory_search
evaluation
transform
notification
human_input
```

12.5 Idempotency

workflow 啟動請求應支援 **Idempotency-Key** header。

如果同一使用者對同一 workflow 傳送相同的 idempotency key, backend 應回傳現有 workflow run，而不是建立重複執行。

13. Governance 設計

13.1 風險等級

使用標準風險等級：

```
low
medium
high
critical
```

含義範例：

```
low      = read-only, internal, reversible
medium   = internal update, limited impact
high     = external action, customer-visible, sensitive data
critical = financial, legal, destructive, production-impacting
```

13.2 Governance Policy 範例

Policy 範例：

```
{
  "name": "Require approval for external emails",
  "conditions": {
    "tool_category": "email",
    "recipient_type": "external"
  },
  "action": "require_approval",
  "reviewer_role": "Manager",
  "risk_level": "high"
}
```

Policy 範例：

```
{
  "name": "Block restricted memory for general agents",
  "conditions": {
    "resource_type": "memory",
    "sensitivity": "restricted",
    "agent_department_mismatch": true
  },
  "action": "deny",
  "risk_level": "critical"
}
```

13.3 Governance Actions

Governance check 結果：

```
allow
deny
require_approval
require_evaluation
require_redaction
```

14. Knowledge 與 Memory 設計

14.1 Knowledge 匯入流程

1. User uploads document
2. Backend stores original file in object storage
3. Backend creates KnowledgeDocument
4. Backend enqueues indexing job
5. Worker extracts text
6. Worker chunks text
7. Worker creates embeddings
8. Worker stores chunks and vector references
9. Document status becomes indexed

14.2 Retrieval Pipeline

1. User, agent, or workflow requests search
2. Backend checks permissions
3. Backend filters by organization, department, sensitivity, ACL
4. Backend performs vector search
5. Backend optionally reranks results
6. Backend returns allowed results only
7. Backend writes audit event for sensitive access

14.3 Memory 規則

Memory 預設不應全域可用。

每次 memory lookup 都必須考慮：

- organization
- user
- agent
- workflow
- department
- sensitivity
- expiration
- policy

15. Agent Runtime 設計

15.1 Agent 執行原則

agents 必須在 backend 控制的邊界內執行。

backend 控制：

- which agent can run
- which workflow can call the agent
- which tools the agent can use
- which memory the agent can access
- which knowledge the agent can retrieve
- which outputs require evaluation
- which actions require approval

15.2 Agent 輸入合約

每次 agent 執行都應接收結構化輸入：

```
{
  "agent_id": "agent_123",
  "workflow_run_id": "run_123",
  "step_id": "step_123",
  "input": {},
  "context": {
    "user_id": "user_123",
    "organization_id": "org_123",
    "allowed_tools": [],
    "allowed_memory_scopes": [],
    "risk_level": "medium"
  }
}
```

15.3 Agent 輸出合約

每個 agent 都應回傳結構化輸出：

```
{
  "status": "completed",
  "output": {},
  "usage": {
    "input_tokens": 1000,
    "output_tokens": 500,
    "cost": 0.0
  },
  "metadata": {
    "model": "provider_model_name",
    "duration_ms": 1000
  }
}
```

16. Security 設計

16.1 必需的安全控制

backend 必須實作：

- HTTPS in production
- password hashing
- JWT expiration
- refresh token rotation
- API key hashing
- permission checks
- request validation
- rate limiting
- file upload validation
- malware scanning if available
- audit logs
- secret management
- CORS restrictions
- security headers

16.2 Prompt Injection 防護

由於 agents 可能會讀取文件及使用工具，因此 backend 應防範 prompt injection。

建議控制措施：

- separate trusted system instructions from untrusted retrieved content
- mark retrieved content as untrusted
- prevent retrieved text from granting tool permissions
- require policy checks before tool calls
- require approval for external or destructive actions
- log tool requests
- evaluate suspicious outputs

16.3 資料存取控制

每個查詢都必須包含 organization 範圍限制。

規則示例：

```
A user from organization A must never access organization B data.
```

敏感資源亦必須強制執行：

- department restrictions
- ACL rules
- role permissions
- policy checks

17. Observability 與 Monitoring

17.1 Logs

所有 logs 都應包括：

- timestamp
- request_id
- organization_id if available
- user_id if available
- action
- status
- duration
- error code if failed

17.2 Health Checks

必需 endpoints :

```
GET /api/v1/health
GET /api/v1/health/ready
GET /api/v1/health/live
```

live 檢查 API process 是否正在運行。

ready 檢查依賴項是否可用：

- database
- Redis
- queue
- vector store
- object storage

17.3 Metrics

追蹤：

- request count
- request latency
- error rate
- workflow run count
- workflow duration
- workflow failure rate
- worker job duration
- queue depth
- approval wait time
- document indexing time
- search latency
- evaluation pass/fail rate
- LLM token usage
- LLM cost

18. 部署設計

18.1 本機開發

本機環境應包括：

- API server
- PostgreSQL
- Redis
- worker process
- optional vector database
- optional object storage emulator

本機開發不得使用任何 Docker。

18.2 生產部署

生產環境應分離以下項目：

- API containers
- worker containers
- PostgreSQL
- Redis
- vector store
- object storage
- monitoring

建議的生產服務：

```
api
worker-workflows
worker-knowledge
worker-evaluations
scheduler
postgres
redis
vector-db
object-storage
```

18.3 Environment Variables

必需的 environment variables：

```
APP_ENV
APP_NAME
API_HOST
API_PORT
DATABASE_URL
REDIS_URL
JWT_SECRET
```

```
JWT_ACCESS_TOKEN_EXPIRE_MINUTES
JWT_REFRESH_TOKEN_EXPIRE_DAYS
CORS_ALLOWED_ORIGINS
OBJECT_STORAGE_PROVIDER
OBJECT_STORAGE_BUCKET
VECTOR_STORE_PROVIDER
LLM_PROVIDER
LLM_API_KEY
LOG_LEVEL
RATE_LIMIT_ENABLED
```

可選：

```
SENTRY_DSN
OTEL_EXPORTER_ENDPOINT
SMTP_HOST
SMTP_PORT
SMTP_USERNAME
SMTP_PASSWORD
SSO_CLIENT_ID
SSO_CLIENT_SECRET
```

19. 測試策略

19.1 Unit Tests

測試：

- permission logic
- governance policy evaluation
- workflow state transitions
- schema validation
- memory access rules
- knowledge filtering
- evaluation rules
- audit event creation

19.2 Integration Tests

測試：

- API routes with database
- workflow creation and execution

- approval flow
- knowledge indexing
- memory search
- worker job execution
- audit log creation

19.3 End-to-End Tests

測試完整流程：

- user logs in
- user starts workflow
- workflow requires approval
- reviewer approves
- worker continues
- evaluation passes
- final output is returned
- audit logs are created

19.4 Security Tests

測試：

- unauthorized access
- cross-organization data access
- role permission failures
- invalid tokens
- expired tokens
- restricted memory access
- restricted knowledge access
- tool permission failures

19.5 Load Tests

測試：

- many workflow runs
- many concurrent users
- large document indexing
- vector search latency
- queue backlog behavior

20. 實作計劃

Phase 0: 專案設定

Goal: 建立 backend 專案基礎。

Tasks:

1. Create backend directory.
2. Initialize Python project.
3. Add FastAPI.
4. Add database library.
5. Add migration tool.
6. Add configuration system.
7. Add logging.
8. Add basic health endpoint.

Deliverables:

- backend app starts locally
- health endpoint works
- PostgreSQL connection works
- Redis connection works
- OpenAPI docs available

Acceptance criteria:

GET /api/v1/health returns success.
API starts with one command.
Database migrations can run.

Phase 1: Authentication 與 Users

Goal: 保護 backend。

Tasks:

1. Create user model.
2. Create organization model.
3. Create role and permission models.
4. Implement password hashing.
5. Implement login endpoint.
6. Implement JWT access tokens.
7. Implement refresh tokens.

8. Implement current user dependency.
9. Implement permission checks.
10. Add seed script for first admin user.

Deliverables:

- login works
- authenticated routes work
- users belong to organizations
- basic RBAC works

Acceptance criteria:

Unauthenticated users cannot access protected endpoints.
Users cannot access another organization's data.
Admin can create users.

Phase 2: Audit Logging

Goal: 讓重要操作可追溯。

Tasks:

1. Create audit log table.
2. Add AuditService.
3. Add request ID middleware.
4. Log login events.
5. Log create/update/delete events.
6. Log workflow run events.
7. Add audit search endpoint.

Deliverables:

- audit events are created
- audit logs are searchable
- request IDs connect API logs to audit logs

Acceptance criteria:

Starting a workflow creates an audit event.
Approving a request creates an audit event.
Changing permissions creates an audit event.

Phase 3: Agent 與 Tool Registry

Goal: 註冊 agents 與 tools。

Tasks:

1. Create agent model.
 2. Create tool model.
 3. Create agent CRUD endpoints.
 4. Create tool CRUD endpoints.
 5. Add allowed tools to agents.
 6. Add allowed memory scopes to agents.
 7. Add risk levels.
 8. Add status management.

Deliverables:

- agents can be created
 - tools can be created
 - agents can be activated/disabled
 - tool permissions can be assigned

Acceptance criteria:

Disabled agents cannot be used.
Agents cannot use tools they are not allowed to use.

Phase 4: Workflow Definitions

Goal: 建立及版本化 workflows。

Tasks:

1. Create workflow model.
 2. Create workflow version model.
 3. Create workflow CRUD endpoints.
 4. Add workflow version creation.
 5. Add JSON schema validation for input/output.
 6. Add workflow activation.
 7. Add workflow disabling.

Deliverables:

- workflows can be created
- versions can be created
- active version can be selected
- input schemas are stored

Acceptance criteria:

Only active workflows can be run.
Invalid workflow input is rejected.
Workflow versions are immutable after activation.

Phase 5: Workflow Run Engine

Goal: 透過 workers 執行 workflows。

Tasks:

1. Create workflow_run model.
2. Create workflow_run_step model.
3. Implement start run endpoint.
4. Add queue integration.
5. Add workflow worker.
6. Implement step execution loop.
7. Implement status transitions.
8. Implement cancellation.
9. Implement retry for failed runs.
10. Persist step outputs.

Deliverables:

- workflow runs can be queued
- workers execute runs
- run status updates are persisted
- steps are tracked

Acceptance criteria:

POST /workflows/{id}/runs creates a queued run.
Worker picks up queued run.
Run becomes completed or failed.
Frontend can fetch run status and steps.

Phase 6: Governance 與 Approvals

Goal: 加入風險控制與人工審批關卡。

Tasks:

1. Create governance policy model.
2. Create policy engine.
3. Add pre-run governance checks.
4. Add pre-step governance checks.
5. Create approval request model.
6. Add approval endpoints.
7. Pause workflow when approval is required.
8. Resume workflow after approval.
9. Fail or cancel workflow after rejection.

Deliverables:

- policies can allow, deny, or require approval
- approval requests are created
- workflows pause for approvals
- reviewers can approve or reject

Acceptance criteria:

High-risk tool action creates approval request.
Workflow status becomes waiting_for_approval.
Approval resumes workflow.
Rejection stops workflow.

Phase 7: Knowledge System

Goal: 上載、建立索引及搜尋 knowledge。

Tasks:

1. Create knowledge document model.
2. Create knowledge chunk model.
3. Add document upload endpoint.
4. Store original documents.
5. Add indexing worker.
6. Extract text.

7. Chunk text.
8. Generate embeddings.
9. Store vector references.
10. Add knowledge search endpoint.
11. Enforce ACL and sensitivity filtering.

Deliverables:

- documents can be uploaded
- documents can be indexed
- knowledge can be searched
- restricted documents are protected

Acceptance criteria:

Indexed documents return search results.
Users cannot search restricted documents without permission.
Document indexing failures are visible.

Phase 8: Memory System

Goal: 加入受控的 memory 儲存與檢索。

Tasks:

1. Create memory item model.
2. Add memory CRUD endpoints.
3. Add memory search endpoint.
4. Add memory scopes.
5. Add expiration handling.
6. Add sensitivity levels.
7. Add memory access policies.
8. Connect memory retrieval to workflow steps.

Deliverables:

- memory can be stored
- memory can be searched
- memory access is scoped
- workflows can retrieve allowed memory

Acceptance criteria:

Agents cannot access memory outside allowed scopes.
Expired memory is not returned.
Sensitive memory access is audited.

Phase 9: Evaluation System

Goal: 在完成前檢查輸出。

Tasks:

1. Create evaluation run model.
2. Implement schema evaluator.
3. Implement rule evaluator.
4. Implement policy evaluator.
5. Optional: implement LLM evaluator.
6. Add evaluation step type.
7. Add final output evaluation.
8. Add evaluation API.

Deliverables:

- outputs can be evaluated
- failed evaluations can block completion
- warnings are visible

Acceptance criteria:

Invalid output schema fails evaluation.
Workflow requiring evaluation cannot complete until evaluation passes.
Evaluation results are linked to runs and steps.

Phase 10: 串流更新

Goal: 讓前端可觀察即時 workflow 進度。

Tasks:

1. Add workflow event table or event stream.
2. Emit events during workflow execution.
3. Add SSE endpoint.
4. Stream run status changes.
5. Stream step events.

6. Stream approval events.
7. Stream final output event.

Deliverables:

- frontend can subscribe to run events
- users see real-time progress

Acceptance criteria:

GET /workflow-runs/{id}/stream streams events.
Step completion appears without page refresh.
Run completion appears without polling.

Phase 11: Process Intelligence

Goal: 從 workflow 執行歷史提供分析能力。

Tasks:

1. Add aggregate queries.
2. Add workflow performance endpoint.
3. Add bottleneck endpoint.
4. Add failure analysis endpoint.
5. Add cost analysis endpoint.
6. Add approval delay endpoint.

Deliverables:

- workflow metrics are available
- bottlenecks can be detected
- costs can be analyzed

Acceptance criteria:

Admin can view average workflow duration.
Admin can view most failed workflows.
Admin can view approval wait time.

Phase 12: Hardening 與 Production Readiness

Goal: 為生產環境作準備。

Tasks:

1. Add rate limiting.
2. Add CORS restrictions.
3. Add security headers.
4. Add structured logs.
5. Add metrics.
6. Add error monitoring.
7. Add backup strategy.
8. Add database indexes.
9. Add load tests.
10. Add security tests.
11. Add deployment manifests.
12. Add CI/CD pipeline.

Deliverables:

- production deployment ready
- monitoring ready
- tests passing
- documentation complete

Acceptance criteria:

All critical endpoints have tests.
No protected endpoint is accessible without auth.
Workflow execution survives worker restart where possible.
API documentation is complete.

21. Minimum Viable Backend

首個可用版本應包括：

- authentication
- users
- organizations
- RBAC
- audit logs
- agent registry
- workflow definitions

- workflow runs
- background worker
- basic governance checks
- approval requests
- run status API
- run step API
- basic knowledge upload/search
- OpenAPI docs

MVP 在第一天不需要進階 analytics、進階 memory 或複雜的 process intelligence。

22. 完成定義

當符合以下條件時，backend server 可視為完成首個可投入生產的版本：

1. Frontend can authenticate users.
2. Frontend can list workflows.
3. Frontend can start workflow runs.
4. Backend validates inputs.
5. Backend enforces permissions.
6. Backend creates audit logs.
7. Backend queues long-running work.
8. Worker executes workflow steps.
9. Workflow status can be fetched.
10. Workflow progress can be streamed.
11. Approval gates work.
12. Knowledge search works with access control.
13. Memory access is controlled.
14. Evaluations can block unsafe or invalid outputs.
15. Admin can view audit logs.
16. Admin can view workflow performance.
17. API documentation is generated.
18. Tests cover critical flows.
19. Deployment is documented.
20. Secrets are not hardcoded.

23. 最終 Backend 目的聲明

Backend API Server 把 `structure.md` 中的架構轉化為可執行平台。

它提供一個安全、受治理、可審計且可擴展的層，讓前端應用程式可以與 agents、workflows、memory、knowledge、governance、evaluations 及 process intelligence 互動。

前端應保持簡單。

backend 應負責強制執行 intelligence、control、security 與 trust。

最終系統關係：

Frontend

- > Backend API
- > Services
 - > Governance
 - > Workflow Engine
 - > Agent Runtime
 - > Knowledge and Memory
 - > Evaluation
 - > Audit
- > Database / Queue / Vector Store / LLM / Object Storage